

Parallel Computing
Parallel Computing Applied to Conway's
Game of Life

Lucas Chanet

Erasmus student

04/06/2026

Contents

1 Introduction	3
1.1 Conway's Game of Life	3
1.2 Project Objectives	3
2 Theoretical Background and Literature Review	3
2.1 CUDA and OpenMP Implementations of Conway's Game of Life (Eric Biggers)	4
2.1.1 Grid Representation and Double-Buffering	4
2.1.2 OpenMP Thread Overhead on CPUs	4
2.1.3 CUDA Block Constraints and Kernel Launches	4
2.2 Development of mixed mode MPI / OpenMP applications (Lorna Smith and Mark Bull)	4
2.2.1 Ease of Implementation vs. Communication Overhead	5
2.2.2 The Bottleneck of Loop Synchronisation Barriers	5
2.3 Fast Simulation of Conway's Game of Life using Bitwise Parallel Bulk Computation on a GPU (Fujita, Nakano, and Ito)	5
2.3.1 Bitwise Parallel Bulk Computation (BPBC)	5
2.3.2 Optimisation via Warp Shuffle Transfer	5
3 Methodology and Implementation Details	6
3.1 Data Representation and the Sequential Baseline	6
3.2 Shared-Memory Parallelism (OpenMP)	6
3.3 Distributed Hybrid Architecture (MPI + OpenMP)	7
3.3.1 Halo Exchange (Ghost Rows)	7
3.3.2 Intra-Node Parallelism	7
3.4 GPU Acceleration (CUDA)	7
3.4.1 Thread Mapping and Block Configuration	7
3.4.2 Asynchronous Execution Model	7
4 Experimental Results and Performance Analysis	8
4.1 Impact of Problem Size and Granularity	8
4.2 Computational Throughput Analysis (MLUPS)	9
4.3 Thread Scalability and Amdahl's Law Limitation	10
4.4 Observation of Super-Linear Speedup	11
4.5 Impact of Iteration Count: OpenMP vs. Hybrid Approach	11
4.6 Massive Acceleration via CUDA Data Parallelism	12

5 Comparative Synthesis and Discussion	12
5.1 Direct Quantitative Comparison with the research of Biggers (2011) .	12
5.2 Structural Comparison with Smith & Bull approach	13
5.3 Advanced GPU Optimizations and Performance Comparison (Fujita, Nakano and Ito)	13
6 Conclusion	13
7 Sources	13

1 Introduction

This project presents an implementation of Conway's Game of Life using parallel computing paradigms with OpenMP, MPI and CUDA. The goal is to analyze how to optimize the game of life using parallel computing.

1.1 Conway's Game of Life

The Game of Life is a cellular automaton invented by John Conway in 1970. It is a "zero-player game", meaning that its evolution only depends on its initial state. No human interaction is needed after the simulation starts.

The simulation runs on a two-dimensional grid where each cell is either alive (1) or dead (0). At each step, the next state of a cell is calculated based on its 8 neighbors using four simple rules:

- **Underpopulation:** A live cell with fewer than two live neighbors dies.
- **Survival:** A live cell with two or three live neighbors stays alive.
- **Overpopulation:** A live cell with more than three live neighbors dies.
- **Reproduction:** A dead cell with exactly three live neighbors becomes alive.

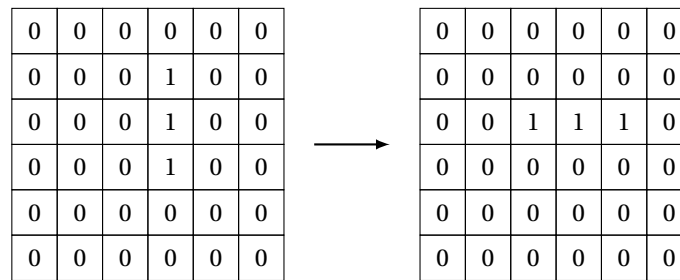


Figure 1: One-step simulation of the Game of Life.

1.2 Project Objectives

The main objective of this project is to implement and compare four different versions of the program to reduce execution time:

1. A baseline **sequential** version.
2. A parallel CPU version using **OpenMP**.
3. A hybrid parallel CPU version using **OpenMP and MPI**.
4. A parallel GPU version using **NVIDIA CUDA**.

To ensure that our parallel code is correct, we verify the results at the end of each simulation. Every implementation (with its given grid) must find the exact same final result because we always used the same random seed.

2 Theoretical Background and Literature Review

To understand the challenges of parallelizing cellular automata, we studied three important research papers. This section presents their main contributions and explains how they directly relate to our project.

2.1 CUDA and OpenMP Implementations of Conway's Game of Life (Eric Biggers)

To establish the technical choices of our project, we study the baseline algorithm and parallel limitations described by Biggers (2011). His work evaluates how Conway's Game of Life runs on both multi-core CPUs (OpenMP) and GPUs (CUDA), highlighting the main performance bottlenecks. It's the starting point of the project.

2.1.1 Grid Representation and Double-Buffering

Biggers (2011) explains that the most intuitive way to store the Game of Life grid is to use a byte array, where alive cells are set to 1 and dead cells to 0. The algorithm requires two copies of the grid (called a double-buffering strategy). Since all cells must update simultaneously, this layout ensures that we read the current state from the first grid and write the new state into the second grid, preventing threads from overwriting data that neighboring cells still need to read.

2.1.2 OpenMP Thread Overhead on CPUs

On a CPU, the loop can be easily parallelized by adding the `#pragma omp parallel for` directive. This automatically splits the matrix rows among the available CPU threads.

However, Biggers (2011) emphasizes a major trap regarding thread management: creating and destroying threads inside the main loop at every single generation creates massive software overhead. To achieve good performance, threads must be created only once before the generation loop starts. They can then persist across all generations using lightweight synchronization. If the grid is too small (e.g., 50×50), the time spent synchronizing the threads is larger than the actual calculation, making a multi-threaded execution slower than a single-threaded baseline.

2.1.3 CUDA Block Constraints and Kernel Launches

On a GPU, the programming model changes completely because loops are removed from the core code. Instead, a CUDA kernel is written for a single cell, and the hardware maps every individual cell to a single lightweight thread. These threads are organized into blocks, typically of size 16×16 .

The primary hardware limitation identified by Biggers (2011) is that CUDA threads belonging to different blocks cannot communicate or synchronize with each other. Because a cellular automaton strictly requires all cells to finish the current generation before anyone starts the next one, a global synchronization is mandatory. The only reliable way to enforce this is to completely exit the kernel and launch it again from the CPU for each generation. While this guarantees correct results, launching a kernel introduces a small time delay. For small grids, this launch overhead makes the GPU slow, but for large grids, the massive parallel power of the GPU easily compensates for it.

Connection to our project: Our implementation directly follows the guidelines and solutions validated by Biggers (2011):

- **Data Structure:** We used a continuous 1D flat array of `unsigned char` combined with a double-buffering system to avoid memory conflicts between threads.
- **GPU Synchronization:** Our CUDA code does not use internal block synchronization; instead, we run the kernel 500 times iteratively from a loop on the CPU host.
- **Workload Scale:** By testing a wide range of grid sizes, from small to a massive 8000×8000 grid, our project demonstrate that the synchronization and kernel launch overheads become negligible as the workload increases, allowing us to fully exploit the GPU's hardware acceleration.

2.2 Development of mixed mode MPI / OpenMP applications (Lorna Smith and Mark Bull)

To better understand how multi-core CPUs handle the Game of Life, we analyze the research by Smith and Bull (2001). They specifically used Conway's Game of Life as a case study to evaluate parallel programming performance on shared-memory architectures.

2.2.1 Ease of Implementation vs. Communication Overhead

Smith and Bull (2001) show that parallelising the Game of Life with OpenMP loop directives is very simple, requiring only 15 extra lines of code compared to around 100 extra lines for an MPI implementation. Their performance results demonstrate that pure OpenMP performs more effectively than MPI on fine-grain problem sizes, such as small grids. This happens because the MPI version is heavily slowed down by the communication overhead of exchanging halo data (boundary cell updates) between different processes.

2.2.2 The Bottleneck of Loop Synchronisation Barriers

However, the authors point out a major limitation in standard loop-based OpenMP implementations. Because directives are placed around separate, consecutive loops within the main generation cycle, they force an implicit synchronisation barrier at the end of each loop region. This forces all CPU threads to stop and synchronise three times within every single generation step. This added synchronisation creates load imbalance issues and significantly reduces the scaling efficiency of the CPU. When mixing MPI and OpenMP in a hierarchical way, this internal thread synchronisation overhead can become so high that the code achieves no significant performance advantage over a pure MPI code.

Connection to our project:

- **Overcoming Small-Scale Bottlenecks:** Smith and Bull (2001) observed that small grids suffer heavily from parallel overhead and loop synchronisation. By testing massive grids such as 8000×8000 for our project, we ensure that the actual computation time per thread is large enough to make these synchronization barriers negligible.
- **Understanding CPU Scaling Limits:** When we increase the number of CPU threads, the speedup is not perfectly linear. The internal synchronization barriers at the end of our OpenMP loop directives create a natural hardware ceiling, confirming that managing thread synchronization is the main bottleneck on multi-core CPUs.

2.3 Fast Simulation of Conway's Game of Life using Bitwise Parallel Bulk Computation on a GPU (Fujita, Nakano, and Ito)

To explore the performance capabilities of GPUs, I analyze the work of Fujita, Nakano, and Ito. This research proposes an approach that is radically different from standard implementation: instead of processing each cell individually, it exploits the hardware's binary calculation capabilities to drastically increase the simulation's throughput.

2.3.1 Bitwise Parallel Bulk Computation (BPBC)

The key concept introduced by the authors is Bitwise Parallel Bulk Computation (BPBC). Instead of representing each cell with a byte (as in our Biggers-based implementation), the authors use 32- or 64-bit words to store the states of multiple adjacent cells within a single integer register.

This technique allows for computing the state of 32 or 64 cells simultaneously using only bitwise logical operations (AND, OR, XOR, SHIFT). This significantly reduces the number of instructions required to update the grid and minimizes access to the GPU's global memory, transforming a costly conditional logic operation into an ultra-fast hardware-level arithmetic operation.

2.3.2 Optimisation via Warp Shuffle Transfer

Beyond bit manipulation, the authors address the shared memory bottleneck. They utilize the `__shfl` instruction (*Warp Shuffle*), a feature specific to NVIDIA architectures that allows threads within the same *warp* to transfer data directly from other threads' registers.

By combining this technique with a "multi-step" simulation the authors manage to drastically reduce the overhead associated with kernel launches and synchronization. This allows data to be kept in the GPU's registers for as long as possible, where bandwidth is at its highest.

Connection to our project:

- **Comparative approach:** Although our implementation relies on the standard programming model, this study helps us understand the theoretical limits of our code. Our choice of a simple `unsigned char` structure prioritizes readability and portability across different architectures, while the BPBC approach represents the most advanced techniques in GPU-specific hardware optimization.
- **Future evolution:** This research shows that if we were to optimize our implementation for even more massive grids (like $10,000 \times 10,000$), switching to a "bitboard" structure (storing bits rather than bytes) would be the most effective lever for performance, albeit at the cost of significantly higher code complexity.

3 Methodology and Implementation Details

This section outlines the data structures, algorithmic paradigms, and parallelization strategies engineered to simulate Conway's Game of Life. The simulation was implemented and evaluated across four distinct paradigms: a sequential baseline, a shared-memory model (OpenMP), a distributed hybrid model (MPI + OpenMP), and a massively parallel model (CUDA).

3.1 Data Representation and the Sequential Baseline

For the Sequential, OpenMP, and CUDA implementations, the cellular grid is linearized into a contiguous one-dimensional array of bytes to maximize spatial cache locality and eliminate the overhead of pointer indirection. A cell's 2D coordinate (r, c) is mapped to a 1D index using the formula:

$$\text{Index}(r, c) = r \times \text{COLS} + c \quad (1)$$

The simulation employs a **double-buffering** mechanism. Two buffers are allocated: `current_grid` and `next_grid`. Instead of performing a costly $\mathcal{O}(N \times M)$ deep copy at the end of each iteration, the host merely swaps the memory pointers:

Listing 1: Pointer swap implementation for double buffering.

```
1 unsigned char *temp = current_grid;
2 current_grid = next_grid;
3 next_grid = temp;
```

3.2 Shared-Memory Parallelism (OpenMP)

The pure OpenMP implementation accelerates the sequential baseline by distributing the nested loops among the available CPU threads, configured explicitly to 2, 4, 6 or 8 threads via `omp_set_num_threads(8)` depending on the processor's capabilities.

A critical optimization implemented here is the use of the `collapse(2)` clause. Rather than exclusively parallelizing the outer loop (which would restrict the number of parallel tasks to $\text{ROWS} = 2000$), merging the row and column iterations exposes a massive, unified iteration space of $2000 \times 2000 = 4,000,000$ independent tasks.

Additionally, for the periodic progress tracking (counting alive cells every 50 iterations), a parallel reduction is utilized to securely accumulate the local counts into a global sum without mutex locks or data races:

```
1 #pragma omp parallel for reduction(+ : current_alive)
2 for (int i = 0; i < ROWS * COLS; i++) {
3     current_alive += current_grid[i];
4 }
```

3.3 Distributed Hybrid Architecture (MPI + OpenMP)

To scale the application across distributed memory environments, a hybrid MPI and OpenMP approach is utilized. Unlike the 1D array approach used in other versions, the MPI implementation leverages a dynamically allocated 2D array (`unsigned char **grid`). This architectural shift significantly simplifies the 1D horizontal domain decomposition (row-wise splitting) across processes.

3.3.1 Halo Exchange (Ghost Rows)

Each MPI process is assigned $\frac{N}{\text{size}}$ rows. To compute the Moore neighborhood of boundary cells, the local sub-grid is padded with two extra rows (index 0 for the top ghost row and `local_rows + 1` for the bottom ghost row). At the beginning of every generation, processes exchange their boundary rows using point-to-point communication:

Listing 2: Halo exchange utilizing MPI_Sendrecv.

```

1 MPI_Sendrecv(current_grid[1], N, MPI_UNSIGNED_CHAR, up_neighbor, 0,
2               current_grid[local_rows + 1], N, MPI_UNSIGNED_CHAR, down_neighbor, 0,
3               MPI_COMM_WORLD, MPI_STATUS_IGNORE);
4
5 MPI_Sendrecv(current_grid[local_rows], N, MPI_UNSIGNED_CHAR, down_neighbor, 1,
6               current_grid[0], N, MPI_UNSIGNED_CHAR, up_neighbor, 1,
7               MPI_COMM_WORLD, MPI_STATUS_IGNORE);

```

3.3.2 Intra-Node Parallelism

Following the halo exchange, each MPI process spawns an OpenMP parallel region to process its local sub-grid. The explicit `#pragma omp parallel for collapse(2) schedule(static)` directive is crucial here.

Because the 1D domain decomposition assigns only a subset of rows to each process, parallelizing solely along the vertical axis would severely limit thread scalability and introduce thread starvation on high-core-count processors. By collapsing the local rows and full columns into a unified block, the hybrid architecture ensures that intra-node CPU cores remain fully saturated with work.

3.4 GPU Acceleration (CUDA)

The CUDA implementation maps the cellular automaton directly onto the GPU hardware. Nested loops are eliminated and replaced by a global thread grid where each thread is responsible for updating a single cell.

3.4.1 Thread Mapping and Block Configuration

The kernel is launched using 2D blocks of 16×16 threads (256 threads per block), aligning with the GPU's warp scheduling (multiples of 32 threads). The execution configuration is computed dynamically to prevent out-of-bounds violations:

```

1 dim3 blockSize(16, 16);
2 dim3 gridSize((COLS + blockSize.x - 1) / blockSize.x,
3               (ROWS + blockSize.y - 1) / blockSize.y);

```

3.4.2 Asynchronous Execution Model

To minimize the PCIe bandwidth bottleneck, data is pushed to the VRAM only once at initialization (`cudaMemcpyHostToDevice`) and pulled back only upon completion (`cudaMemcpyDeviceToHost`).

Inside the iteration loop, the kernel launches are asynchronous from the CPU's perspective. Because all kernel calls are dispatched to the default stream (Stream 0), the GPU guarantees implicit sequential execution. The CPU safely swaps the device memory pointers and queues the next kernel launch without needing an explicit `cudaDeviceSynchronize()` barrier inside the loop, keeping the GPU hardware pipeline saturated and maximizing throughput. Synchronization is only enforced once after the final iteration loop is completed.

4 Experimental Results and Performance Analysis

In this section, we evaluate and analyze the performance of our four implementations of the Game of Life: Sequential, OpenMP, Hybrid (2 MPI Processes $\times$ 3 OpenMP Threads), and CUDA. Benchmarks were conducted on an AMD Ryzen 5 7535HS processor (6 physical cores, 12 threads) and an NVIDIA GeForce RTX 4050 (75W) GPU. To study the behavior of our code regarding scalability and overhead, we tested various grid dimensions (50×50 to 8000×8000) and iteration counts. Performance is quantified using execution time (seconds), Speedup ($S = T_{seq} / T_{par}$), Parallel Efficiency ($E = S / p$), and in Millions of Lattice Updates per Second (MLUPS).



Figure 2: Overall execution time comparison.

As visually demonstrated, the performance gap between the CUDA implementation and the CPU-based methods is significant. The following subsections will provide a detailed technical analysis of these results.

4.1 Impact of Problem Size and Granularity

Our results show that parallel performance varies significantly depending on the size of the grid. By analyzing the workload given to each processor, we show how the amount of computation impacts the overall efficiency of our parallel implementations.

- **Small Grids (50×50 and 100×100):** For the 50×50 grid computed over 1,000,000 generations, the sequential implementation completes in 64.685 seconds. The OpenMP version (6 threads) only achieves a minor speedup of 1.19 (54.099 s), while the Hybrid version performs slightly better at 1.66 (38.834 s). Also the CUDA speedup drastically drops to 8.67 (7.456 s).

This happens because a small grid does not provide enough work for each core or GPU thread. Since there is very little computation to process, the time spent managing parallel execution (creating threads, exchanging border cells, or starting the GPU) becomes larger than the actual computation time. This matches the results of *Eric Biggers*, who showed that when matrices are too small, the lack of workload causes parallel management costs to eliminate any speed benefits.

- **Large Grids (2000×2000 to 8000×8000):** As the domain expands, parallel implementations become hyper-efficient. On a 2000×2000 grid (500 iterations), OpenMP and Hybrid versions reduce the execution time from 46.535 seconds down to ~ 9.30 seconds, achieving a speedup of ~ 5.0 .

The CUDA implementation shows a massive performance jump, achieving a speedup of 800.9 (in 0.058 s) on the 2000×2000 grid, which climbs to a 1378.6 (in 0.675 s) speedup on the 8000×8000 grid. The compute-intensive nature of large grids successfully amortizes the cost of parallel management and saturates the GPU's thousands of processing cores.

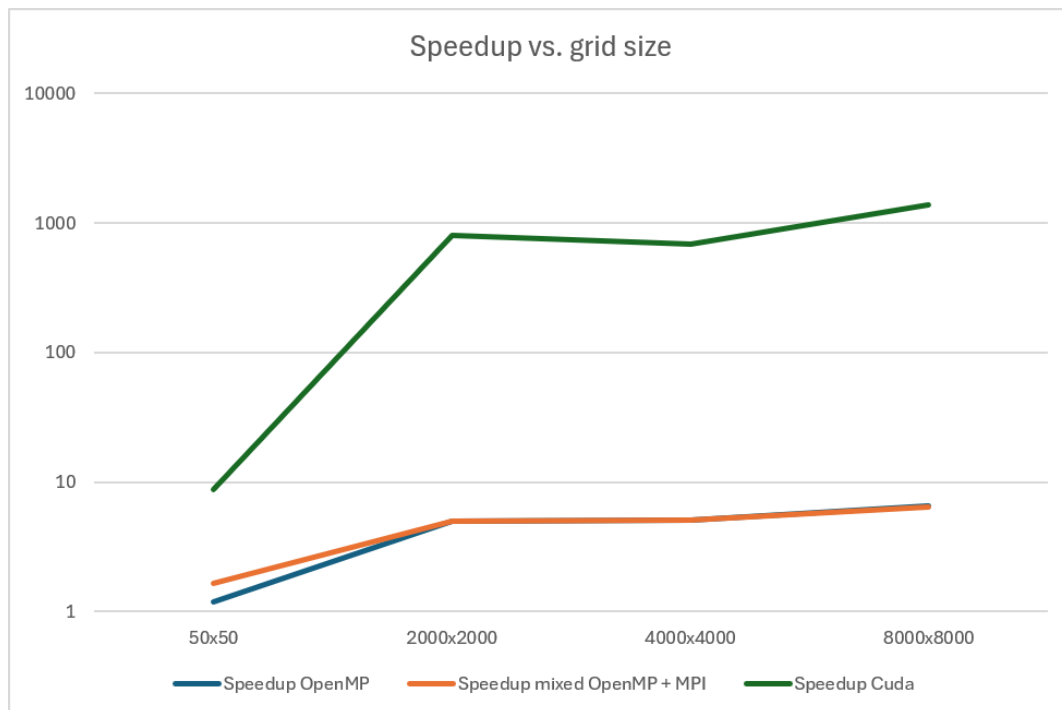


Figure 3: Evolution of speedup across different grid sizes

4.2 Computational Throughput Analysis (MLUPS)

To evaluate the raw processing power independent of absolute execution times, we analyze the computational throughput measured in Millions of Lattice Updates per Second (MLUPS).

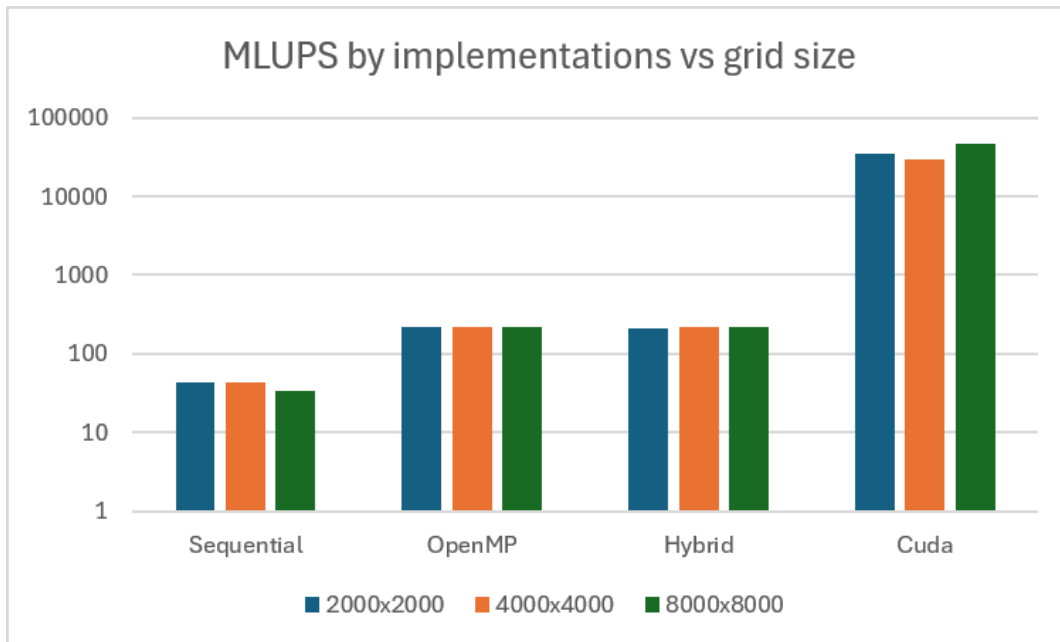


Figure 4: Computational throughput (MLUPS) comparison across large grid sizes

These throughput results clearly show the fundamental difference between CPU and GPU computing:

- **CPU performance saturation:** The sequential code remains strictly limited by single-core clock frequencies, hovering between 34.34 and 43.30 MLUPS. Transitioning to OpenMP (6 threads) and Hybrid ($2P \times 3T$) approaches multi-core scaling limits, plateauing predictably around ~ 214 to 222 MLUPS across all three matrix sizes.
- **CUDA Hardware Saturation:** The CUDA implementation shows a really different performance tier. It delivers 34421.63 MLUPS on the 2000×2000 grid and reaches its peak of **47340.35 MLUPS** on the 8000×8000 domain. This means the NVIDIA RTX 4050 updates over 47 billion cells per second in my environment.

This huge gap happens because the Game of Life spends more time moving data in memory than doing math. The CPU is slowed down by managing threads and loading data row by row. In contrast, the GPU uses its thousands of cores and fast memory (VRAM) to process thousands of cells at the exact same time. On the 8000×8000 grid, there is finally enough work to keep all GPU cores fully busy, making the whole execution incredibly efficient.

4.3 Thread Scalability and Amdahl's Law Limitation

To analyze scalability, performance was tested using 1 to 8 threads on an 8000×8000 grid over 250 iterations.

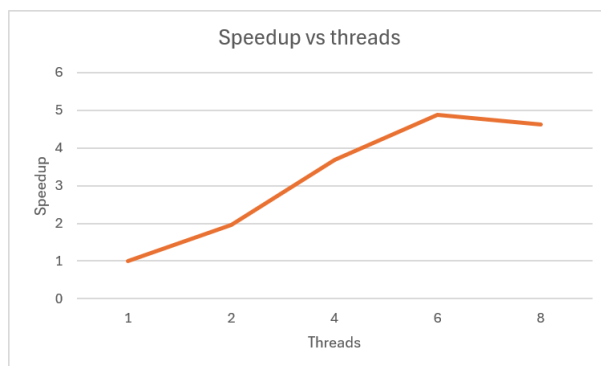


Figure 5: Speedup curve on the 8000×8000 grid.

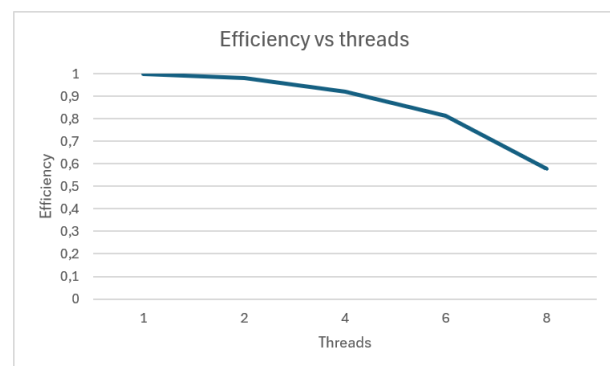


Figure 6: Parallel efficiency on the 8000×8000 grid.

As shown in Figures 5 and 6, the speedup increases steadily up to 6 threads, reaching a maximum of 4.88. However, the performance gains are non-linear, and parallel efficiency decreases continuously from 100% down to 81%. As taught in the course, for a fixed problem size, increasing the number of allocation threads causes the overall efficiency to drop due to the unavoidable growth of parallel overhead (T_O). This behavior illustrates the diminishing returns predicted by **Amdahl's Law**.

At 8 threads, performance actively degrades as the execution time increases to 82.50 s causing the speedup to drop to 4.62 as well as the efficiency. This degradation demonstrates that engaging logical SMT threads introduces resource contention and synchronization overhead.

4.4 Observation of Super-Linear Speedup

An interesting phenomenon occurs during the 8000×8000 grid simulation (500 iterations). The sequential execution requires 931.879 seconds. When utilizing 6 OpenMP threads, the runtime drops to 143.667 seconds, translating to a speedup of **6.48** and a parallel efficiency of **108%**. Similarly, the Hybrid setup (2P $\times$ 3T) achieves a speedup of **6.44** (144.500 s) and an efficiency of **107%**.

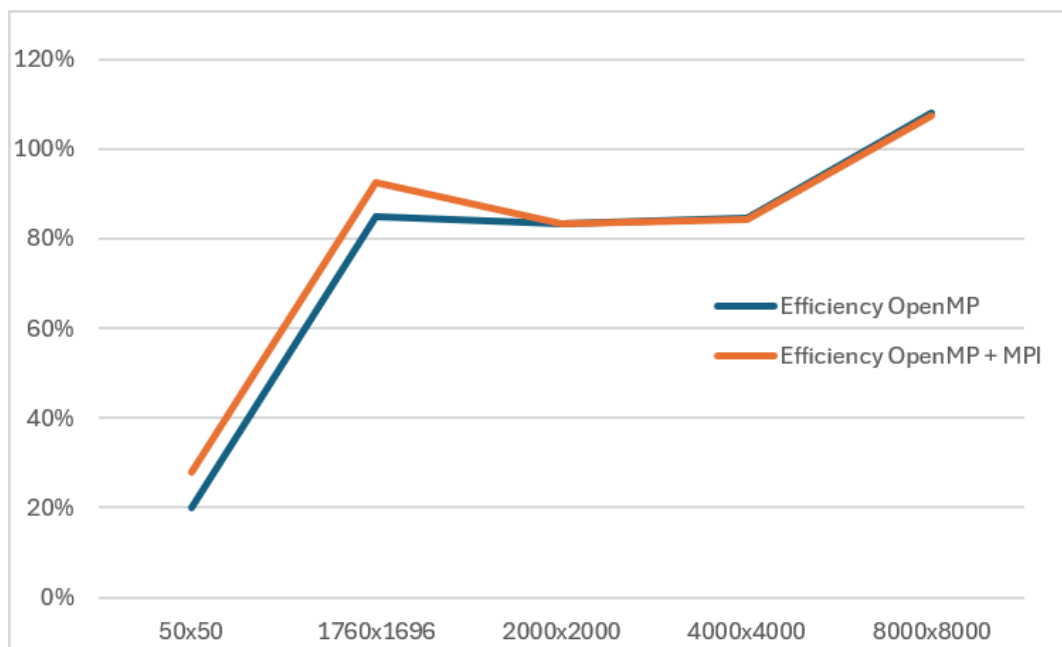


Figure 7: Observation of Super-Linear Speedup with 6 threads for pure OpenMP and 2 MPI processus and 3 OpenMP threads for the mixed implementation

This efficiency exceeding the theoretical 100% limit is known as a **super-linear speedup**.

1. In sequential mode, the large 8000×8000 grid exceeds the CPU's L3 cache capacity, forcing frequent and slow RAM accesses.
2. When using parallel execution (OpenMP or MPI), the grid is split into smaller sub-domains that fit directly into the fast L3 cache of each core.

4.5 Impact of Iteration Count: OpenMP vs. Hybrid Approach

To analyze the long-term behavior of both paradigms, a comparative benchmark was conducted by scaling the number of simulation iterations (from 1 up to **1000000**). The experimental data reveals a critical threshold where the Hybrid MPI/OpenMP configuration outperforms pure OpenMP.

As the simulation scales to 1000000 iterations, the mixed approach shows its real interest

- **Pure OpenMP** execution time increases to 54.099 seconds.

- **Hybrid MPI/OpenMP** demonstrates superior scalability, completing the simulation in 38.834 seconds.

This phenomenon illustrates the critical accumulation of parallel overhead (T_O) over time. In a pure OpenMP setup, every single generation loop forces all threads to synchronize through implicit barriers. Over a million iterations, this synchronization penalty accumulates drastically, causing heavy contention on the shared memory bus.

The Hybrid approach splits the grid into independent MPI domains. By reducing the number of active OpenMP threads per process, loop synchronization barriers remain localized. Although MPI introduces a communication overhead to exchange boundary rows, this overhead is heavily amortized over a high number of iterations, making the Hybrid model much more stable for long-running simulations.

4.6 Massive Acceleration via CUDA Data Parallelism

The CUDA implementation demonstrates the massive performance gains of GPU computing for the Game of Life:

- On the 2000×2000 grid, the runtime drops to 0.0581 seconds achieving a 800.9 speedup.
- On the 8000×8000 grid, the simulation takes only 0.6759 seconds achieving a 1378.6 speedup.

The Game of Life maps perfectly to the GPU's SIMT (Single Instruction, Multiple Threads) architecture because each cell updates independently using only its neighbors' previous states. This allows millions of cells to be processed simultaneously across thousands of CUDA cores.

While researchers like *Fujita, Nakano and Ito* have achieved high performance using advanced software approaches like Bitwise Parallel Bulk Computation (BPBC), our standard CUDA implementation demonstrates that it can achieve comparable multi-fold performance acceleration.

5 Comparative Synthesis and Discussion

5.1 Direct Quantitative Comparison with the research of Biggers (2011)

Table 1: Comparison of execution times (Grid 1760×1696) : My experimental results (Ryzen 5 7535HS + RTX 4050) vs. Biggers (Intel atom + CUDA-capable GPU)

Source	Implementation	Number of Generations			
		1	10	100	1000
My Work	Sequential	0.079 s	0.770 s	7.214 s	69.024 s
	OpenMP (6 threads)	0.020 s	0.177 s	1.492 s	13.562 s
	Mixed (2P $\times$ 3T)	0.0179 s	0.1851 s	1.5082 s	12.4587 s
	CUDA	0.0005 s	0.0011 s	0.0066 s	0.0594 s
Biggers	OpenMP (1 thread)	0.035 s	0.355 s	3.557 s	35.621 s
	OpenMP (2 threads)	0.018 s	0.181 s	1.807 s	18.163 s
	CUDA	0.0100 s	0.1000 s	0.9960 s	9.9630 s

The data in Table 1 highlights a significant performance gap between the hardware architectures used in the 2011 study and the modern Ryzen 5 7535HS platform.

- **Performance Gap:** My sequential implementation runs significantly faster than the reference data, reflecting the massive evolution in IPC (Instructions Per Cycle) and clock speeds over the last decade. While Biggers utilized an Intel Atom processor—a low-power CPU focused on energy efficiency—my implementation leverages a high-performance architecture, resulting in faster raw compute cycles.
- **Scalability Trends:** Despite the gap in raw time, the scalability trends remain consistent. The CUDA implementation, in both studies, provides the most dramatic speedup, confirming the architectural advantage of SIMT (Single Instruction, Multiple Threads) for cellular automata.

5.2 Structural Comparison with Smith & Bull approach

I implemented a hybrid MPI/OpenMP approach for the Game of Life that structurally mirrors the Diffusion Monte Carlo (DMC) algorithm discussed by *Smith & Bull*. Both structures rely on an outer generational loop, an inner multi-threaded phase (updating walkers vs. computing rows with OpenMP), and periodic global synchronization (their block averages vs. my MPI_Reduce every 50 generations).

Interestingly, executing this structure on a small workload (50×50 grid over 1,000,000 generations) revealed a performance gain for the Mixed mode ($2P \times 3T$) compared to the pure OpenMP configuration (38.834 s vs. 54.099 s).

- **Reduced OpenMP synchronization overhead:** With only a 50×50 grid, the computational work per generation is very small. Using 6 OpenMP threads therefore introduces a significant synchronization cost at each iteration. In Mixed mode ($2P \times 3T$), only 3 threads synchronize within each process, reducing this overhead and improving performance.
- **Negligible MPI communication cost:** The halo exchanges involve only 50 bytes per message, making communication extremely lightweight.

5.3 Advanced GPU Optimizations and Performance Comparison (Fujita, Nakano and Ito)

My CUDA implementation achieves really good performance, completing 1,000 generations on a 1760×1696 grid in just 0.0594 s, which corresponds to a $1,162\times$ speedup compared to the sequential baseline. These results can be directly put into perspective with the advanced GPU optimization techniques proposed by *Fujita, Nakano, and Ito*.

- **Raw Hardware Power vs. Bitwise Optimization (BPBC):** Fujita, Nakano and Ito rely on the BPBC (*Bitwise Parallel Bulk Computation*) technique to pack 32 or 64 cells into a single memory word (integer) to maximize bit-level parallelism. Thanks to the architecture of my modern GPU (RTX 4050), my code achieves good performance using a standard mapping (one thread per cell).
- **Kernel Launch Overhead Reduction:** To alleviate the high cost of launching CUDA kernels at each generation on older GPUs, Fujita, Nakano and Ito developed a multi-step simulation within a single kernel. As we saw in my benchmarks, the execution time remains perfectly linear and extremely low.

6 Conclusion

In conclusion, this comparison highlights two main points:

- **The Speed of CUDA:** The GPU (RTX 4050) is by far the fastest solution, running up to 1,800 times faster than the normal sequential code. Modern hardware is so powerful that the time lost starting the calculations on the card is not even noticeable. This allows us to get excellent performance with a very simple code.
- **CPU Parallelism:** My results show that there is no massive difference between pure OpenMP and Mixed mode (MPI/OpenMP). However, both implementations are obviously much faster than the normal sequential code.

The efficiency of a parallel program running the game of life depends on finding the right balance between the grid size, the hardware power, and the time lost in synchronization.

7 Sources

- **Toru Fujita, Koji Nakano, and Yasuaki Ito.** "Fast Simulation of Conway's Game of Life using Bitwise Parallel Bulk Computation on a GPU".
URL: https://www.cs.hiroshima-u.ac.jp/cs/_media/life-ijfcs.pdf
- **Lorna Smith and Mark Bull.** "Development of mixed mode MPI/OpenMP applications". ResearchGate.
URL: https://www.researchgate.net/publication/2432870_Development_of_mixed_mode_MPIOpenMP_applications

- **Eric Biggers.** *"CUDA and OpenMP Implementations of Conway's Game of Life"*. Macalester College / CSinParallel.
URL: <https://selkie.mcalester.edu/csinparallel/dev/modules/GameOfLife/source/FromEric/final/paper/paper.pdf>